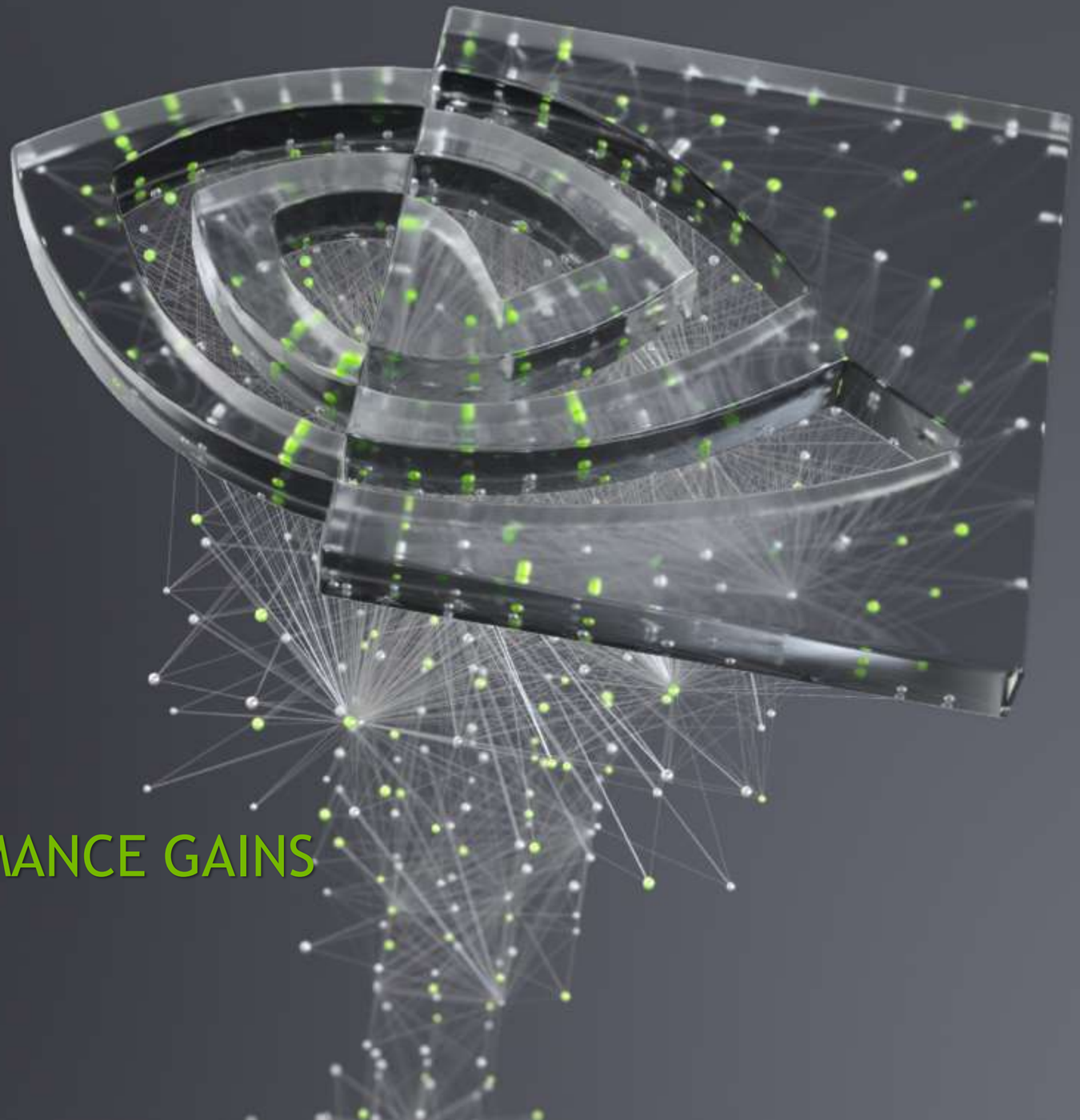


KTLS HW OFFLOAD

IMPLEMENTATION AND PERFORMANCE GAINS

Tariq Toukan, Tal Gilboa and Bar Tuaf, August 2020



AGENDA

Motivation & Introduction

Layers & Responsibilities

Data Flows

Performance – Expectations and Measurements

Summary

MOTIVATION

TLS is used everywhere

- More than 80% of all web pages are HTTPS servers using TLS protocol. [1]
- Constant growth in accessing web servers requires increasing rate of bandwidth.
- Offloading computational networking tasks, thereby freeing CPU resources to serve more users and process more data.
- Relying on the robust and resilience Linux networking stack.

[1] 14-day average measured at July 29 2020, reported by Mozilla Firefox - https://letsencrypt.org/stats/#growth

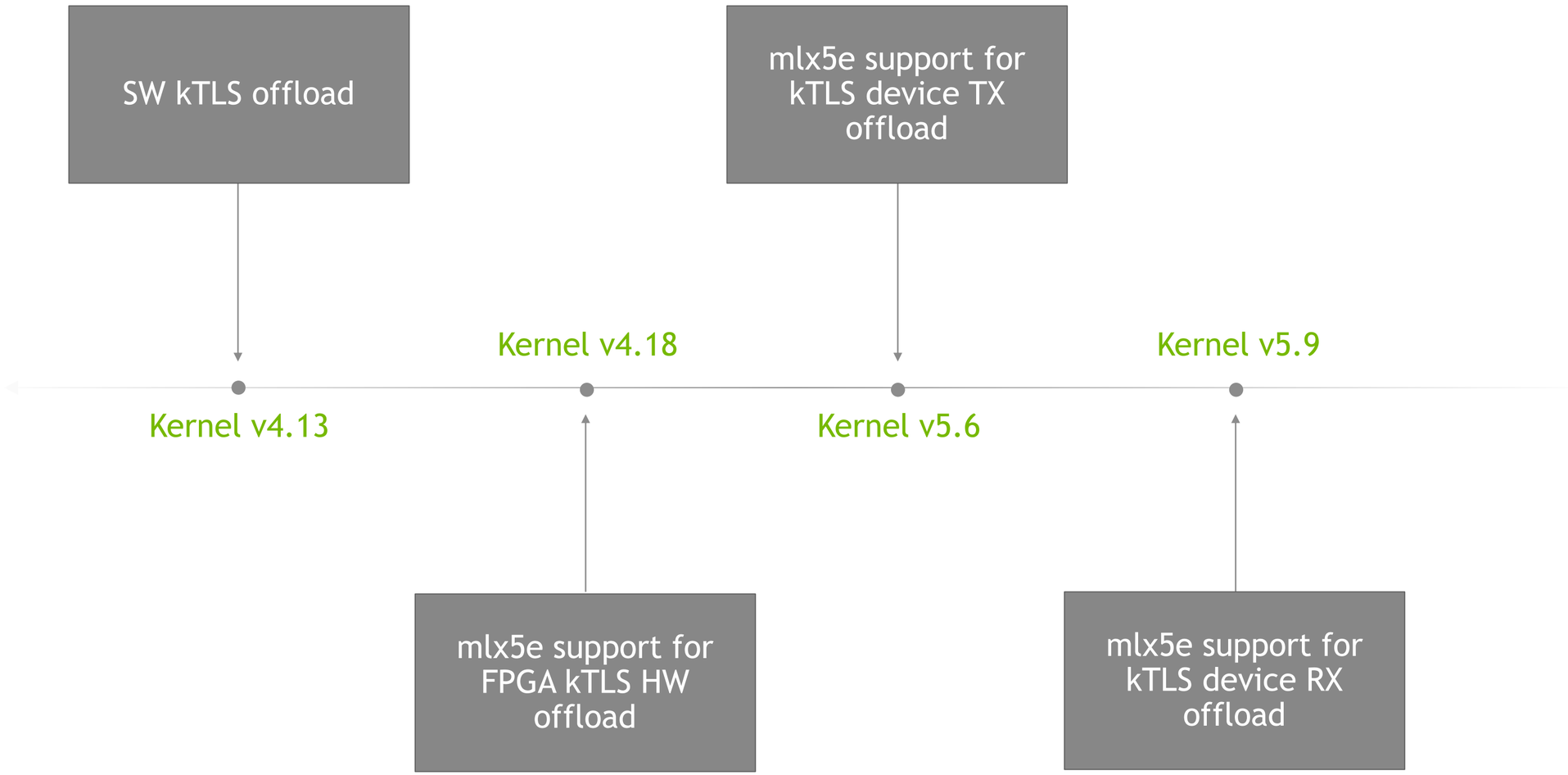
NVIDIA.

INTRODUCTION

- TLS – Transport Layer security, we used TLS version 1.2
- Layer 4 - on top of TCP traffic
- TLS process was done by user space applications
  - SW KTLS offload into kernel
  - HW kTLS offload crypto crunching to the device
- This presentation will cover
  - kTLS device data flows
  - Synthetic and real life application results

NVIDIA.

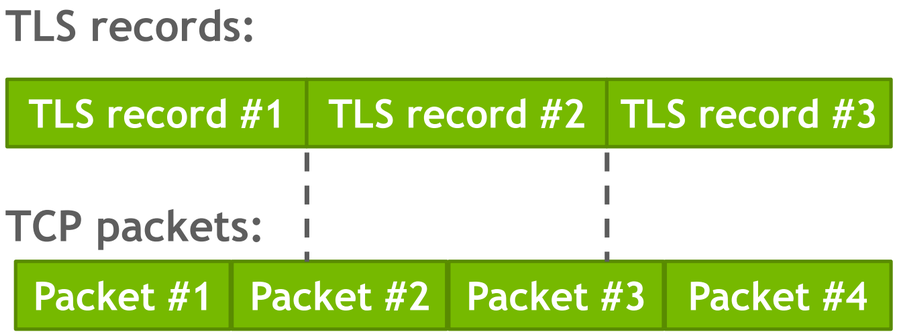
KTLS TIMELINE



HW OFFLOAD CHALLENGES

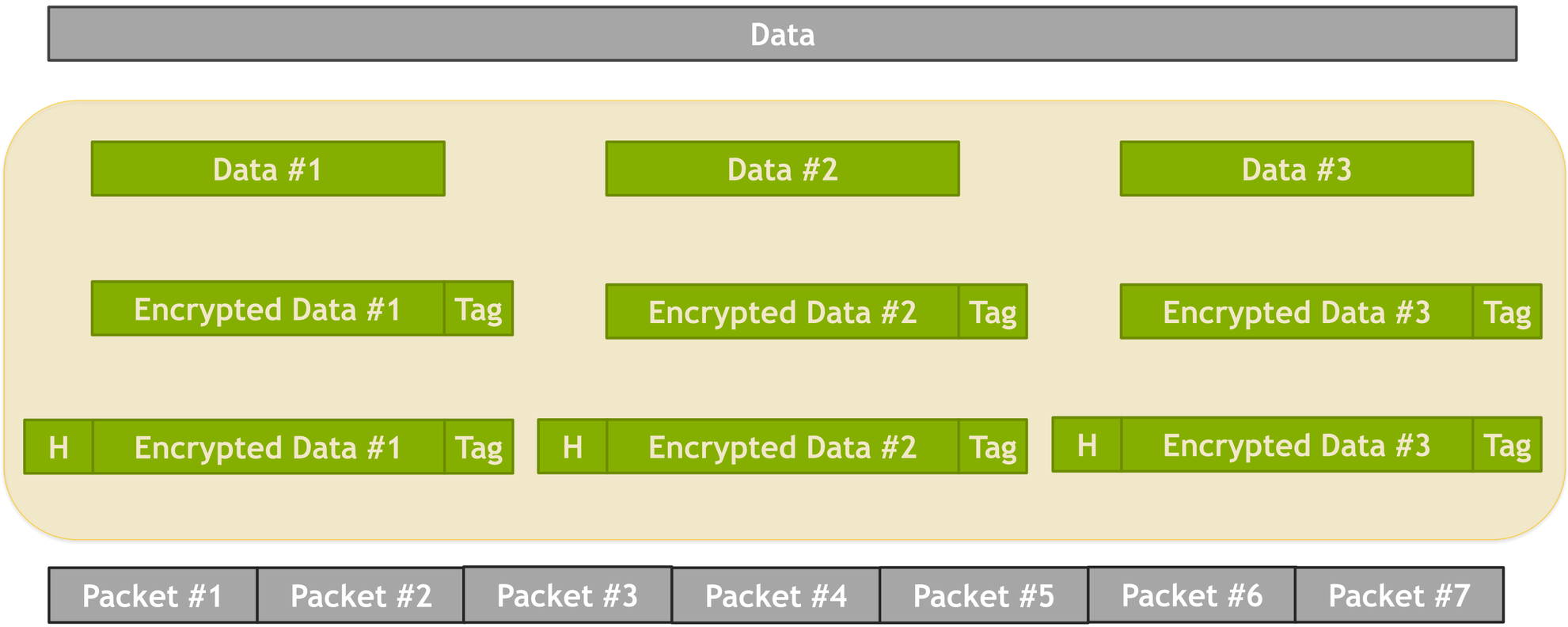
Getting kTLS device to work

- Process records not packets
  - TLS record can spread on several TCP packets
- AES counter mode - Generates keystream using IV (Initialization Vector) and key
  - XORing plaintext with proper keystream = Ciphertext
- Driver initialize HW context on creation
  - Context is provided by TLS stack
- Tracking TLS record state
  - HW must track it for subsequent packets
  - TLS stack coordinate with driver to restore HW state whenever needed



TLS LAYERS ROLES

- User space-
  - Application data
- kTLS module-
  - Fragmentation
  - Encrypt, Decrypt & auth
  - Records
- TCP-
  - MSS packets

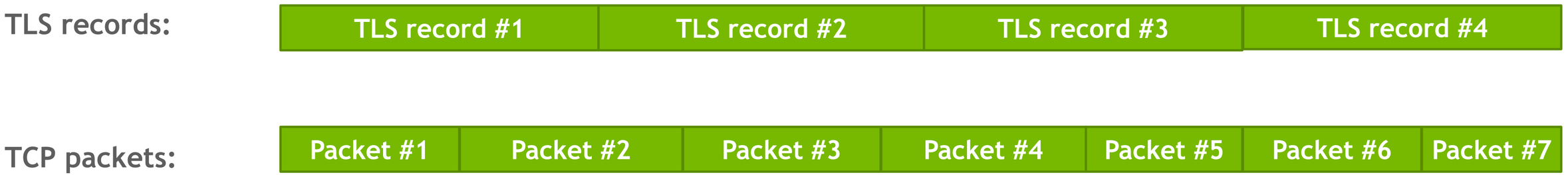


H – TLS headers Tag – TLS record authentication Tag

TX PATH

In order

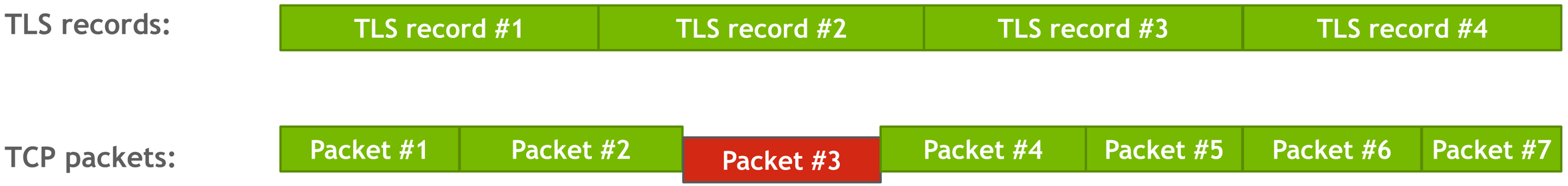
- Check if packet belongs to a TLS offload socket- True
- Check packet TCP sequence number against the expected TCP sequence number – True
- Prepare the TX descriptor with the needed context information
- Send the packet for authentication and encryption in the NIC



TX PATH

Resynchronization flow

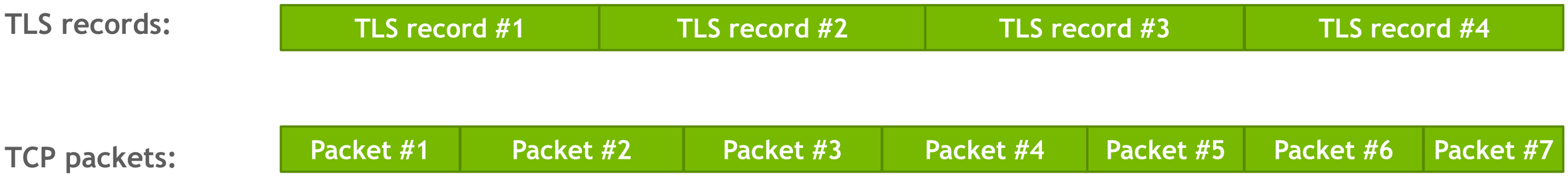
- Check if packet belongs to a TLS offload socket- True
- Check packet TCP sequence number against the expected TCP sequence number - **False**
  - Trigger **resync** flow, driver does fast path communication to sync the HW context
  - kTLS stack is not involved
- Prepare the TX descriptor with the needed context information
- Send the packet for authentication and encryption in the NIC



RX PATH

In order

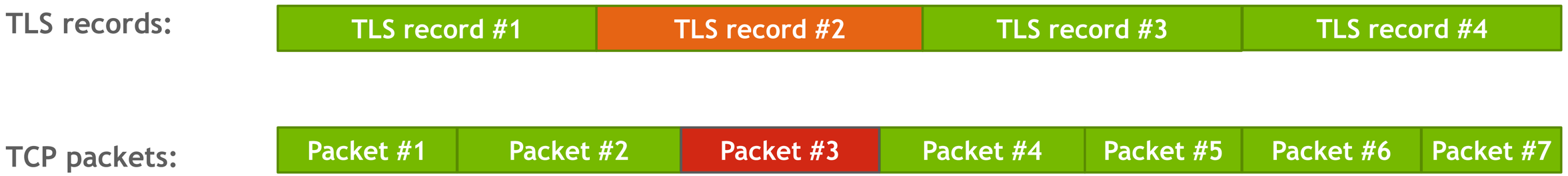
- HW is in offload state, it decrypts the connection packets and indicates it in the RX descriptor
- Driver marks skb->decrypted and hands it to the stack
- Stack checks each TLS record:
- All packets belong to record are decrypted - True
- Copy plaintext data to user space



DRIVER RX PATH

Partial decryption

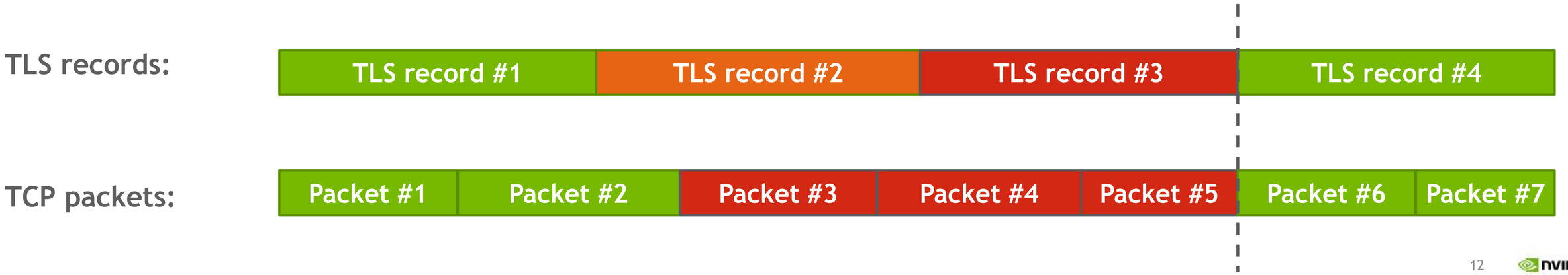
- When HW TLS encounter a drop/out-of-order in the middle of a record it will not lose his state since this packet does not contain TLS headers
- Stack checks each TLS record:
- All packets belong to record are decrypted – **False**
  - Part of the record is decrypted – True
    - **Partial decryption:** Use SW to decrypt the ciphered packets
- Copy plaintext data to user space



RX PATH

Resynchronization flow

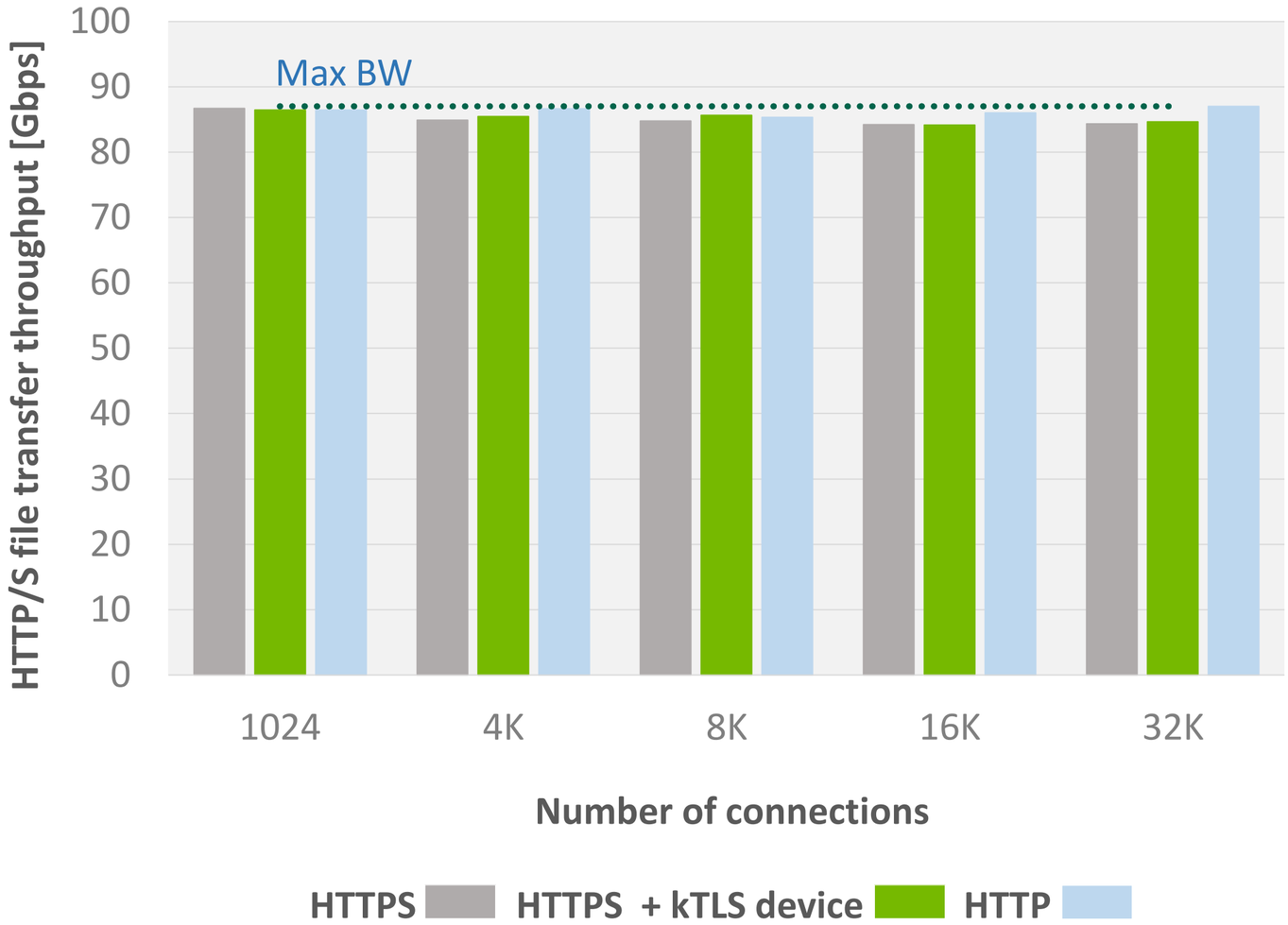
- Out of order or packet drops might get the HW out of sync and to stop decrypting packets
- It indicates a resync request in the RX descriptor to the device driver
- Driver notifies the kTLS stack via the async resync API
- Later on the driver provides the guessed TCP sequence to the kTLS stack after querying it from the device
- If the TLS stack finds it matches any of the incoming packets that it tracks, notify the driver which resync the device
- Meanwhile SW TLS stack decrypts all these packets
- Once successfully resynced, the device will return to offload state and decrypt the incoming connection packets



REAL LIFE APPLICATION

- System-
  - Two AMD EPYC 7742 processors
  - Connected thorough switch with ConnectX-6Dx NIC
- Nginx server and Wrk benchmark – 1K to 32K connections
- kTLS device TX offload only
- Comparison between:
  - HTTPS + SW kTLS
  - HTTPS + kTLS HW offload
  - HTTP
- All cases use sendfile(), Nginx sendfile-chain which still copies the data to protect from changes in the page cache

- **Throughput = line rate (100Gbps)**
- **Graph reports Wrk application layer throughput - ~85Gbps (Goodput – without headers)**



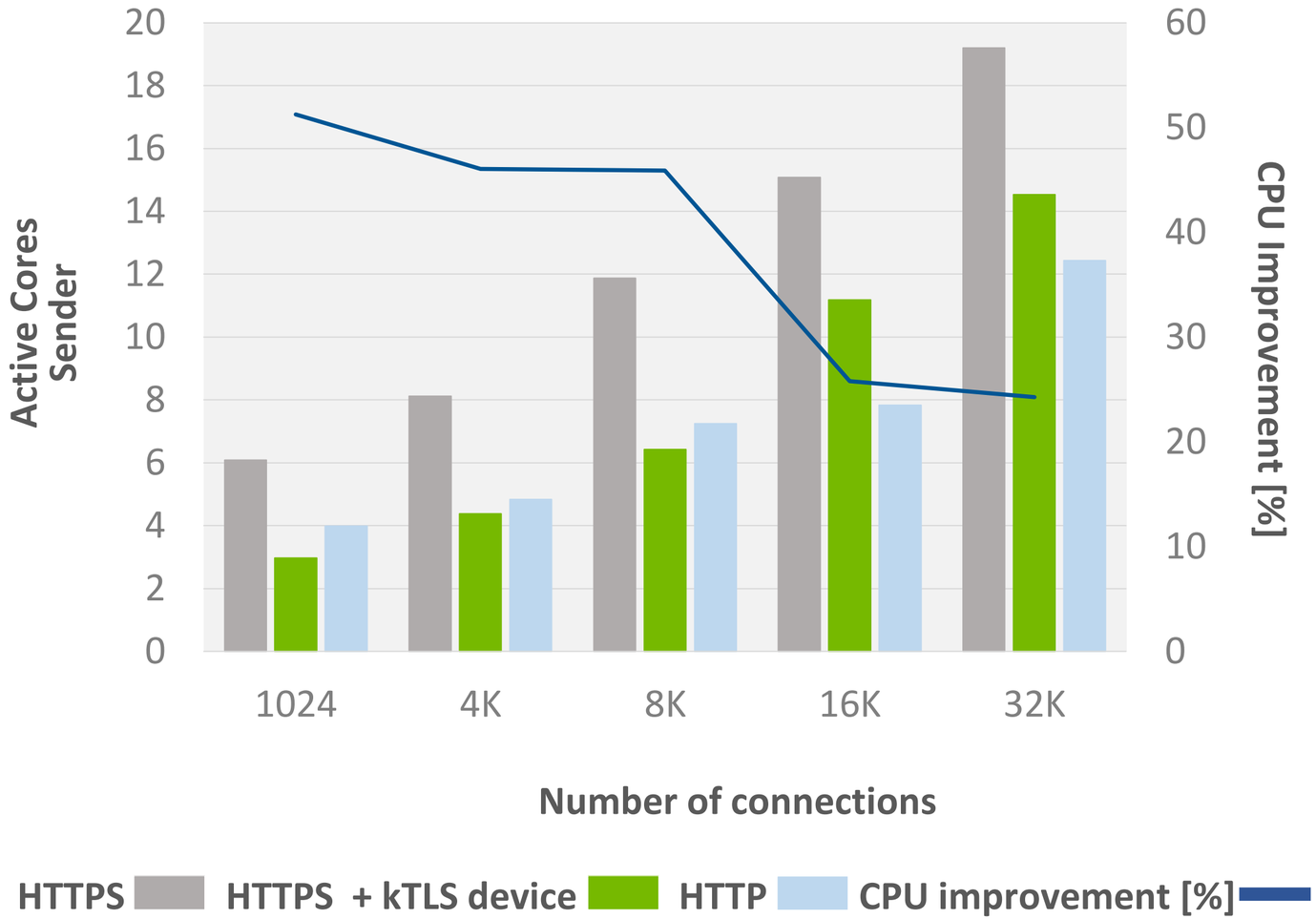
REAL LIFE APPLICATION

- Active cores = sum of the CPU utilization of all system cores
- CPU improvement formula:

$$\frac{\textit{https active cores} - \textit{https kTLS device TX active cores}}{\textit{https active cores}} = \textit{CPU Improvment}$$

- Significant CPU improvement **of up to 50%** less active cores

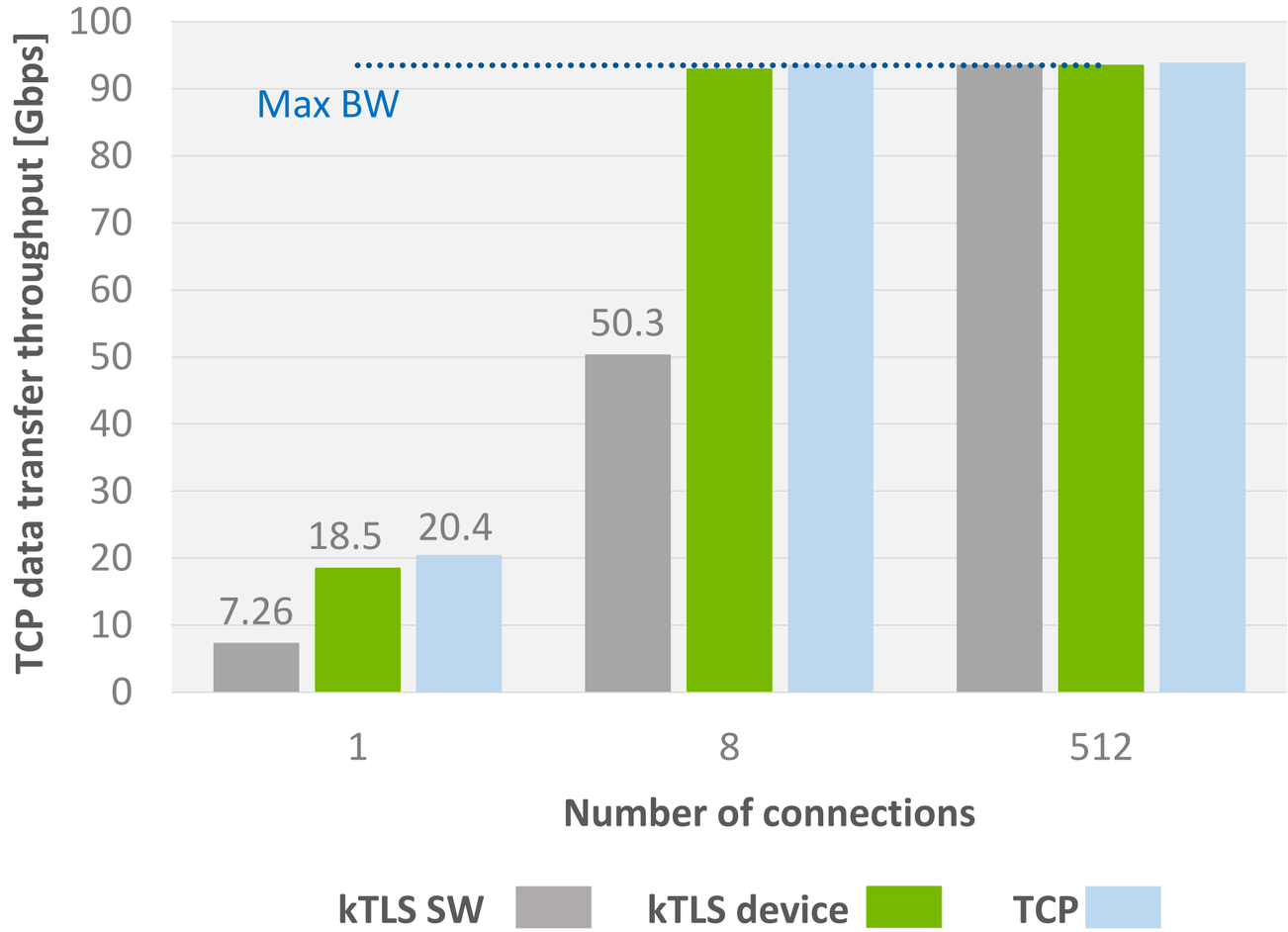
- Note: 1K to 8K cases shows higher packet rate using HTTP in comparison to HTTPS traffic using kTLS device which leads to higher CPU utilization



FULL UNIDIRECTIONAL DEVICE OFFLOAD

- System-
  - Two Intel Xeon E52687W 3.00GHz systems
  - Connected back-to-back with ConnectX-6Dx NIC
- Patched version of Iperf 2 with support for TLS handshake using OpenSSL libraries
- Comparison between:
  - SW kTLS using TLS send and receive methods
  - kTLS offload using TLS send and receive methods
  - TCP using stack send and receive methods

- **Full unidirectional device offload - TX offload on the server and RX offload on the client**
- **2.5x speed up!**
- **Get's line rate with less connections**
- **When line-rate achieved gain is in CPU utilization.**
- **Graph reports Iperf application layer throughput - ~94.5Gbps (Goodput - without headers)**

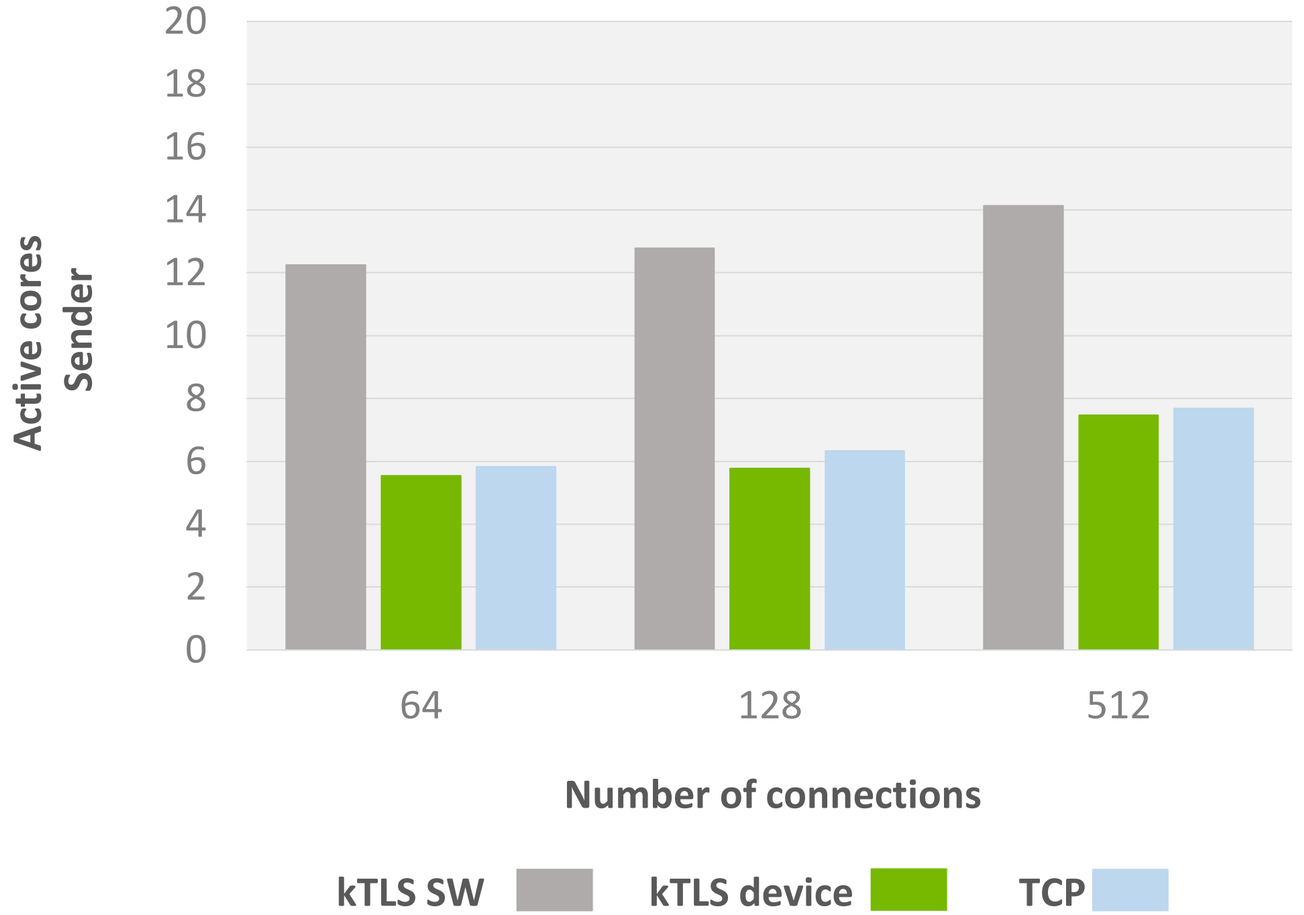


FULL UNIDIRECTIONAL DEVICE OFFLOAD

Sender Side

- **Throughput = line rate (100Gbps)**
- **24 cores**
- **Saves CPU power and recovers “all” of SW kTLS CPU overhead**

- Note: higher packet rate leads to higher CPU utilization for TCP traffic in comparison to TLS traffic using kTLS device



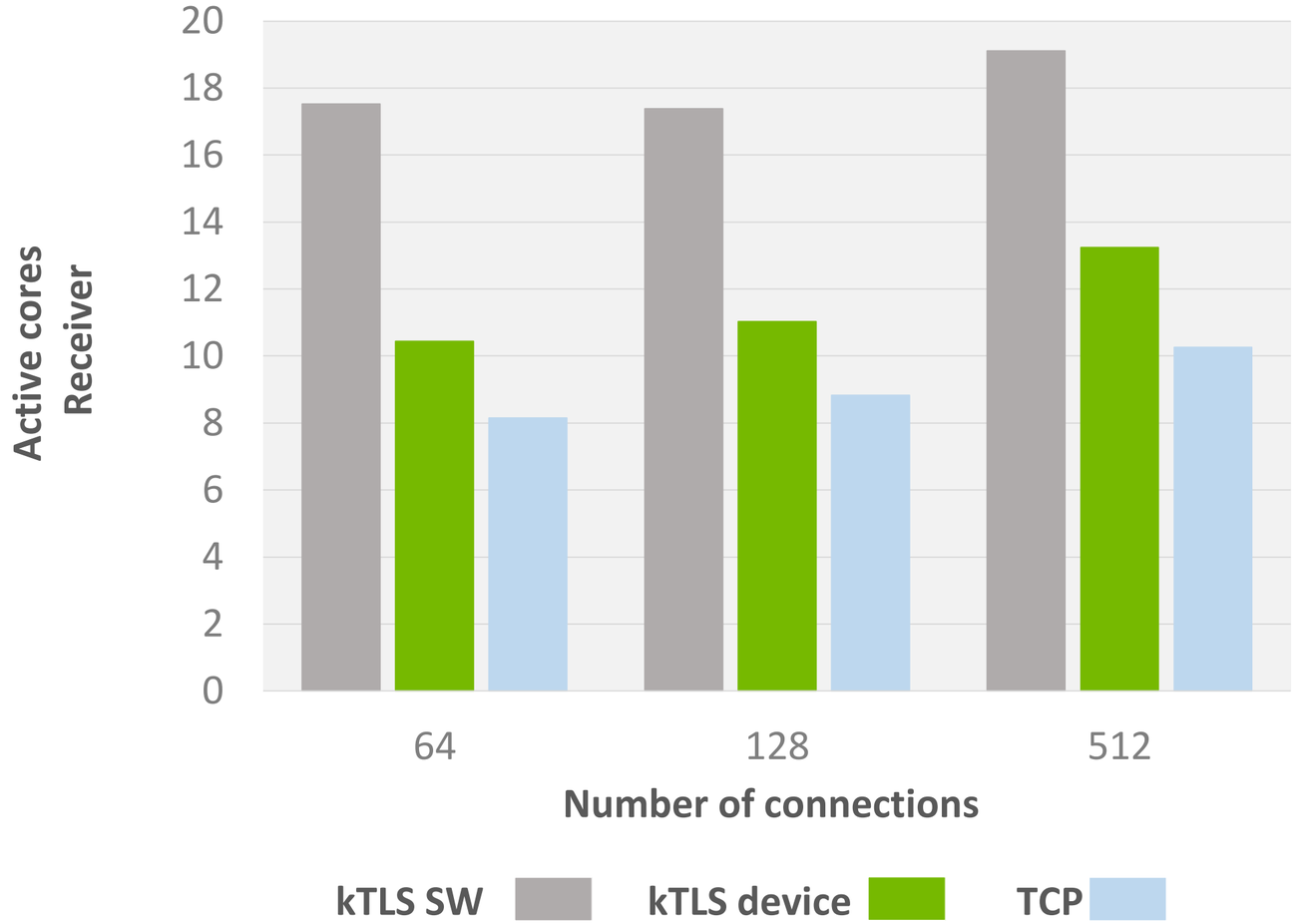
Source: Source information is 14 pt, italic

FULL UNIDIRECTIONAL DEVICE OFFLOAD

Receiver Side

- **Throughput = line rate (100Gbps)**
- **24 cores**
- **Saves CPU power and recovers 66%-75% of SW kTLS CPU overhead**

- 512 connections-
  - TCP use 10 active cores to achieve maximum BW
  - SW kTLS use 19 - 9 more than TCP
  - kTLS device offload use 13 active cores - 3 more than TCP = 66% CPU recovery from TLS overhead



Source: Source information is 14 pt, italic

BENEFITS

Using TLS device API

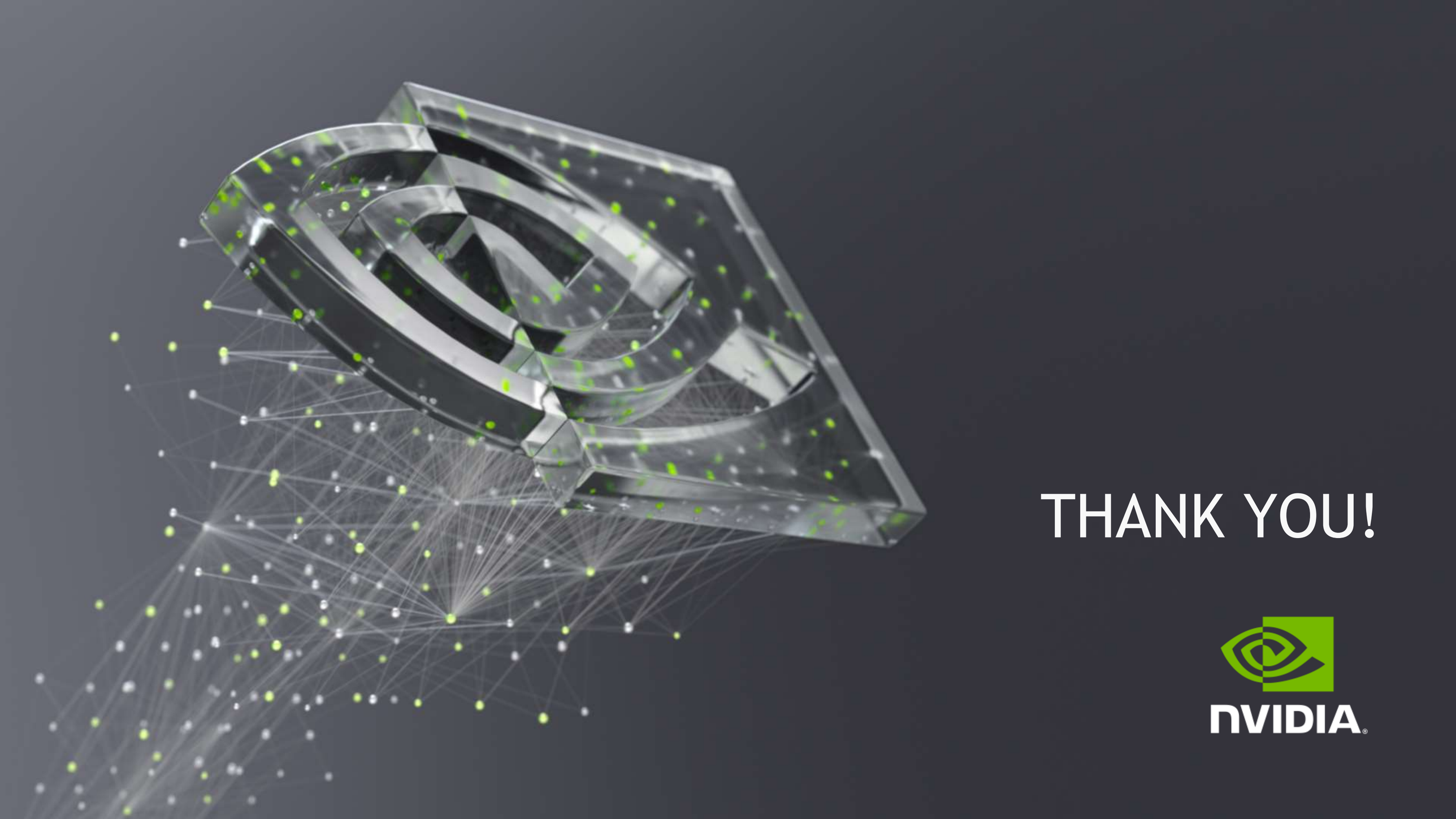
- “Inline operation”- saves PCIe bandwidth and PCIe lanes in the server as data goes through the PCI only once.
- Crypto data is not written to RAM.
- Software Stack remains unchanged with non-crypto flow, TCP/IP, memory management and congestion control.
- Software is involved when HW loses its state.

NVIDIA.

SUMMARY

- Significant performance throughput speed-up gained by offloading crypto processing to the network device
- Recovers from significant percentage of the CPU overhead caused by TLS protocol
- Prevent hackers from getting the data before it is encrypted since non-crypted data is not written to RAM
- Easy to maintain, high-performing TLS implementation, relying on the great networking stack

NVIDIA.



THANK YOU!

NVIDIA